

三 AI 自主项目开发系统

Socratic Mentor · Autonomous Executor · Independent Controller

版本：V1.3（需求与治理架构基线）

文档状态：方案说明 / 待实施验证

编制日期：2026 年 9 月 29 日

适用对象：项目所有者、系统架构师、Agent 开发人员、测试与运维人员

文档目的：将已确认的三 AI 协作体系转化为可开发、可审计、可验收的系统设计基线。

执行摘要

本系统由项目所有者、指导老师 AI（Mentor）、执行工程师 AI（Executor）、调度监管 AI（Controller）及确定性控制程序组成。用户通过结构化苏格拉底式交流确定目标与重大取舍；Mentor 在授权范围内规划、下达任务和独立验收；Executor 严格按版本化契约实施并可提交技术改进提案；Controller 独立审计需求与任务完整性，调度状态、检查权限和证据，并通过确定性程序实施门禁。

设计重点不是增加 AI 数量，而是实现可追溯需求、明确权责、受控变更、真实成果验证、复杂问题的分层交互和长期自主恢复。系统不得以多个 AI 意见一致代替客观验证，也不得以自动化为理由越过用户授权。

文档目录

1. 目标、范围与设计原则
2. 角色与权力矩阵
3. 分层授权与项目宪章
4. 苏格拉底式需求澄清规范
5. Controller 独立完整性审计
6. 两级用户决策与重大冲突
7. 项目分解与版本化任务契约
8. 执行规范与技术提案 RFC
9. 独立验收、返工与回归
10. 调度状态机、异常与恢复
11. 共享工作区、追溯和长期记忆
12. 安全、资源、权限与高风险操作
13. 里程碑集成与最终交付

14. 技术实现建议与部署架构

15. 分阶段实施与系统级验收

附录 A—F：模板、状态、权限、审计清单、场景与术语

1. 目标、范围与设计原则

1.1 建设目标

- 以尽量少且高质量的用户交互获得完整、准确、可验收的需求。
- 让 AI 在明确授权范围内自主完成架构规划、细分任务、开发、测试、审查、返工和交付。
- 保证任何任务可追溯到用户确认的需求，任何完成声明有实际证据。
- 允许执行者主动创新，但技术变更必须先获得适当审批。
- 在复杂变化发生时暂停受影响任务，通过两级决策机制让用户作出知情决定。
- 支持长周期项目的状态持久化、故障恢复、成本约束和阶段性集成验收。

1.2 适用边界

适用于软件、嵌入式、电子工程及其他可拆分为明确交付物的项目。对于物理设备、生产系统、安全关键功能和需要真实环境验证的成果，自动化验证不等于现场或人工验收。第一版优先采用单一正式执行者，后续扩展并行执行时必须增加文件锁与合并策略。

1.3 不可违反的设计原则

1. 用户拥有项目目标、核心需求、重大风险和超授权变更的最终决定权。
2. 导师在用户委托范围内拥有唯一技术决策和验收责任；执行者不得自我验收。
3. 监管者可以基于明确规则阻断流程，但不得自行改写需求或选择技术方案。
4. AI 意见、文字总结与真实证据分离；以实际文件、可复现测试和授权记录为准。
5. 所有正式任务均由版本化契约约束；未经批准的变更不能追认成合规。
6. 高风险工具操作依赖程序化权限隔离，而非仅依赖提示词。
7. 未验证必须明确标为未验证；没有证据不能宣称通过。

2. 角色与权力矩阵

角色	拥有的职责与权力	明确禁止
用户 / Owner	确认宪章、核心目标、重大取舍、超授权变更、必要的最终实物验收	不要求持续介入普通任务
Mentor / 指导老师	需求引导、需求解释、架构、里程碑、任务契约、RFC 审批、独立验收、统一对用户提问	擅改核心需求；直接在执行者正式分支上改代码；跳过证据
Executor / 执行工程师	实现、调试、测试、真实报告、提出 RFC、登记风险和技术债	扩大授权；修改受保护验收标准；批准自身提案或验收自身成果
Controller / 调度监管	完整性审计、任务调度、状态机、版本锁、证据/权限检查、预算监控、阻断违	技术裁决、擅自解释用户意图、直接向执

	规	行者发布技术方案
确定性控制程序	权限执行、文件路径白名单、状态跃迁、 签发校验、审计日志、资源配额	以模型输出替代不可绕过的规则

2.1 单一责任原则

技术方案由 Mentor 负责；正式实现由 Executor 负责；流程合法性由 Controller 负责；核心需求与重大取舍由用户负责。Controller 的完整性审计是流程门禁而不是技术方案的第二决策中心。

2.2 对监管 AI 的制约

Controller 的每条阻断意见必须标注审计规则、对应证据、影响范围及补救条件。Mentor 可提交反证；客观问题通过工具和规则复核，需求解释争议按两级用户决策处理。监管者不得因“理论上还可继续问”而无限阻断。

3. 分层授权与项目宪章

层级	文件/责任	规则
L0	用户确认的项目宪章	项目目标、核心功能、硬约束、重大风险、预算上限、交付标准、用户授权边界
L1	导师制定的架构和里程碑	必须追溯 L0；涉及 L0 变更时需用户批准
L2	导师签发的任务契约	必须符合 L0/L1；指定文件、范围、禁止事项、验收和预算
L3	执行者工作计划/RFC	不得覆盖上级契约；获批 RFC 通过新契约版本生效

所有文件应具有 ID、版本、状态、生效时间、批准主体、关联需求和历史变更记录。用户口头回答经 Mentor 总结并由用户确认后，才转化为受控核心需求；低风险授权内细节可由导师记录决定依据。

4. 苏格拉底式需求澄清规范

4.1 何时启动

项目启动；新里程碑存在关键未明确需求；重大变化/冲突达到二级决策；用户主动要求重新澄清；Controller 审计发现阻碍当前工作的重要需求缺口。

4.2 八个覆盖维度

维度	需要澄清的核心内容	输出
目标与真实意图	问题背景、用户价值、成功定义、优先级	目标陈述与成功指标
功能与场景	用户、主流程、输入输出、异常与边界	功能需求与用例
技术与约束	硬件、软件、接口、平台、兼容性、现有资产	硬约束/偏好/可委托项
质量与性能	实时性、可靠性、精度、安全、维护性	可量化非功能指标
范围与禁区	必须做、暂不做、明确禁止、阶段边界	范围基线
资源与周期	预算、设备、工具、人员、时间	资源约束与优先级

交付与验收	源代码、硬件、文档、测试、演示	交付清单和可验证标准
风险与权限	失败容忍度、回滚、外部依赖、授权边界	风险表和升级规则

上述维度必须评估是否适用，但不要求逐项机械提问。嵌入式项目可按需细化供电、实时性、EMC、通信和保护；软件项目可细化数据、并发、部署和隐私。

4.3 单个问题的五项准入检查

- 8. 必要性：答案是否影响实际需求、架构、任务边界、验收或重要决策？
- 9. 已知性：是否已经有可靠且仍有效的答案？不得重复询问。
- 10. 责任归属：是用户需求决策还是导师应自行处理的普通技术细节？
- 11. 时机：现在不回答是否妨碍当前架构或里程碑？可延后的明确登记。
- 12. 表达：是否具体、中立、背景充分、不诱导、不一次混合多个无关决定？

不符合任一条件的问题，应由导师自行解决、引用已有结论、登记延后或重新表述；不得把无意义提问传给用户。

4.4 五阶段澄清法

- 13. 明确真实目标：从使用场景与成功标准开始，而非预设技术方案。
- 14. 确定功能与约束：划清必须、可选、禁止和可委托内容。
- 15. 关键问题追问：针对风险、边界、反例、异常场景及架构敏感因素。
- 16. 覆盖与冲突复核：核对八维矩阵、需求间矛盾、未证实假设和依赖。
- 17. 结构化总结确认：向用户呈现已确认需求、假设、延后项、禁区与验收标准。

4.5 停止与继续条件

仅当当前阶段关键需求已明确、阻塞性冲突已解决、验收可定义、未决问题有责任人和澄清时点、用户确认关键总结，且 Controller 完整性审计通过时，才能冻结相应需求。非关键的未来细节可带记录延后，不必为了形式完整无限追问。

5. Controller 独立完整性审计

5.1 审计对象与时点

审计对象包括原始对话的有效需求记录、导师总结、八维覆盖矩阵、冲突清单、需求版本、里程碑与任务契约。强制审计时点：初始需求冻结前、新里程碑开始前、小节契约发布前、重大变更生效前；必要时增加阶段性抽查。

5.2 七类核查

- 18. 范围完整性：所有适用维度是否被评估，关键场景是否遗漏。
- 19. 提问有效性：有无重复、无关、诱导、过早或由导师推卸责任的问题。
- 20. 需求准确性：每项关键结论是否能回溯用户原话或正式授权。
- 21. 逻辑一致性：需求、接口、性能、预算和时间之间是否冲突。
- 22. 澄清深度：当前阶段的架构敏感问题是否足够明确。

- 23. 边界完整性：必须做、暂缓、禁止、可委托项是否明确。
- 24. 可验收性：是否具备可测量、可观察或可人工核实的完成条件。

5.3 审计输出与权限边界

结论	含义	处理
PASS	当前阶段覆盖充分，无阻塞性缺口	允许进入下一流程
GAP	存在明确且影响当前工作的遗漏	列出缺口、证据和建议问题，退回导师
CONFLICT	需求或契约存在实质矛盾	导师分析并按授权处理或升级用户
DEFER	存在不影响当前工作的未决事项	登记负责人、触发条件和最迟澄清节点

Controller 只能提出“缺少什么以及为什么重要”，不能代替用户填入答案；所有缺口报告应说明对设计、执行或验收的具体影响。

6. 两级用户决策与重大冲突

6.1 一级：快速决策

适用条件：问题单一、选项及后果清楚、依赖关系已分析、用户一句话即可作出知情决定。Mentor 提供背景、选项、关键影响和推荐的中立技术分析（不替用户决定），用户确认后更新记录和契约。

6.2 二级：苏格拉底式深度澄清

适用条件：多个相互依赖的问题；单个重大且复杂的问题；原架构失效；核心需求冲突；用户无法依据简短背景直接决定。导师先汇总证据和影响图，再围绕相关八维需求逐层提问，最终形成备选方案、取舍、受影响资产和待用户确认的变更包。

6.3 自动升级与防拆分规避

一级交流暴露新的重大依赖、用户表示需要深入讨论或无法说明关键后果时，自动升级二级。不得将本应整体分析的复杂冲突拆成多个简单问题规避澄清。二级问题充分澄清后，可以用一次简明最终确认结束。

6.4 变更期间的隔离

Controller 暂停受影响任务、锁定当前版本、保存分支与运行状态；不受影响且已授权的任务可继续。导师完成影响分析，检查需求、接口、代码、测试、文档及后续里程碑；用户批准后发布新基线并触发必要回归。

7. 项目分解与版本化任务契约

7.1 规划层级

项目宪章 → 系统架构 → 里程碑 → 小节任务 → 工作计划 → 实际交付 → 验收证据。每层对象拥有稳定 ID，维护向上追溯关系及向下实现/测试关系。

7.2 标准契约字段

字段	约束说明
task_id / version / status	唯一编号、不可覆盖的版本号、受控状态
requirement_refs / milestone_id	对应原始需求与里程碑
objective / prerequisites	本节目标与已完成的前置条件
must_do / allowed_paths	必须实现内容及允许修改的路径/工具
bounded_scope	只做到什么程度，明确暂缓的优化与非必要功能
forbidden_scope	禁止修改的文件、接口、依赖及禁止操作
deliverables / acceptance	逐项可核验的成果、测试、日志与质量阈值
budget / risk / rollback	时间/调用/成本预算、风险等级、回滚方式
change_policy / owner	RFC 触发条件、审批人、超授权升级规则

契约由 Mentor 签发，经 Controller 检查完整性、上级一致性、权限和版本冲突后生效。执行者必须明确确认正在执行的 task_id/version。

7.3 任务粒度

一个小节应具备独立成果、明确文件范围、可复现验证及可控回滚。过大的任务继续拆分；过小且不产生独立价值的任务可以合并。禁止为了追求任务数量而制造大量无意义审批。

8. 执行规范与技术提案 RFC

8.1 执行者工作规范

- 25. 读取当前有效契约、相关需求与接口基线，核对前置条件。
- 26. 制定不越界的执行计划，在隔离分支/沙箱完成实施。
- 27. 保留真实工具调用、文件差异、编译/测试日志及未验证事项。
- 28. 发现范围外缺陷时登记风险或技术债，不擅自顺手修改。
- 29. 遇到更优方案、任务冲突或不可满足条件时提出 RFC 或 BLOCKED。
- 30. 提交包含真实变更、测试证据、风险、偏差及回滚点的工作报告。

8.2 RFC 必填项与审批

RFC 包括编号、原契约版本、问题证据、建议方案、备选方案、预期收益、风险、影响范围、资源需求、验证计划、回滚策略和需改变的契约字段。导师可以批准、拒绝、要求补证或批准限时隔离实验。批准必须生成新版本契约，Controller 确认旧版本停止生效后才能继续正式实施。

8.3 争议处理

Executor 可凭新增证据申请一次复议；监管者的程序性阻断也必须附规则与证据。无新证据不得循环争论。超出导师授权、改变核心需求或存在重大风险时交用户决定。

9. 独立验收、返工与回归

9.1 六维验收

维度	检查方式
方向正确性	需求追溯、架构一致性、接口约束
落实完整性	契约逐项核对、实际文件和交付物检查
真实性	真实工具输出、提交记录、日志与可复现验证
言行一致性	报告与 Git diff、实际行为及测试记录比对
质量充分性	单元/集成测试、异常边界、性能及适用静态检查
范围合规性	路径白名单、禁止项、依赖变更和过度设计检查

Mentor 必须独立查看源文件与变更、在适用条件下重新执行验收测试，不能只读 Executor 自述。正式验收测试由独立受保护位置管理，执行者不能擅自削弱。无法在当前环境验证的硬件或现场结果必须标为未验证，并设人工验证节点。

9.2 验收结果

状态	处理
PASS	强制项全部通过，记录证据并允许下一节
REWORK	给出明确失败项与受限返工契约，重新验收
BLOCKED	缺少条件或证据，暂停并指定解阻责任人
ESCALATE	超授权、重大冲突或不可接受风险，进入用户决策

9.3 返工与回归

同一缺陷返工次数达到预算上限，必须根因分析并调整方案或升级，不得无限循环。公共接口、核心依赖或共享模块改变时，导师指定受影响回归测试，Controller 检查执行记录。不得为使当前结果合格而事后降低验收标准。

10. 调度状态机、异常与恢复

10.1 任务状态

DRAFT（草拟）→ AUDITING（监管审计）→ READY（已授权待执行）→ RUNNING（执行中）→ SUBMITTED（待验收）→ ACCEPTED（已验收）。可从 RUNNING/SUBMITTED 进入 REWORK、BLOCKED、CHANGE_PENDING 或 ESCALATED；变更审批完成后通过新版本回到 READY。

10.2 强制门禁

- 31. 没有已批准契约不得启动任务。
- 32. 版本不匹配、超出文件/工具权限或依赖未满足时拒绝执行。
- 33. 执行报告缺少必要证据时不得进入 PASS。
- 34. 变更待批期间不得继续执行受影响的旧任务。
- 35. 任务达到预算、返工次数或风险阈值时触发暂停/升级。

36. 所有状态跃迁由确定性程序校验并写入不可随意覆盖的审计日志。

10.3 故障恢复与幂等

每次任务启动、提交、审批、合并和高风险操作使用唯一事件 ID。Controller 持久化任务状态、有效契约版本、Git 提交、未完成工具操作和预算余量。系统重启后先对账再恢复，避免重复执行不可逆操作。异常中断时优先回滚到已知稳定点；无法确认的外部副作用应人工复核。

11. 共享工作区、追溯和长期记忆

建议使用一个 Git 仓库、独立 Agent 上下文、Executor 工作分支和 Mentor 只读审查工作区。共享“资料”，但不共享未经控制的正式文件写权限。

路径	主要内容
charter/	用户确认的宪章、原始需求、授权记录
architecture/	架构、接口、设计决策记录
milestones/	里程碑与版本化任务契约
proposals/	RFC、实验、审批与复议记录
src/ 与 tests/	正式代码及受保护测试
evidence/	工具日志、测试、差异、审查报告
agent_state/	任务状态、快照、预算、异常、决策日志
delivery/	阶段成果与最终交付包

11.1 需求追溯

为需求、架构决策、任务、代码变更和测试证据分配稳定 ID；维护 REQ → ARC → MILESTONE → TASK → COMMIT → TEST 的双向关联。每次重大变更由 Mentor 生成影响清单，Controller 逐项检查是否更新或安排复验。

11.2 长期记忆与可信来源

Agent 聊天历史不作为唯一事实来源。已确认需求、有效契约、决策日志、验收记录与状态快照是权威项目资料；模型重启先读取权威资料，区分已确认事实、推测和待验证信息。外部文档或执行输出应视为不可信输入，不能借其中的指令改变 Agent 权限。

12. 安全、资源、权限与高风险操作

12.1 最小权限

Executor 只获当前任务必要的文件、网络、命令、凭据和环境权限；Mentor 原则上拥有只读检查及受限测试权限；Controller 拥有调度与门禁权限，但不能替代用户批准高风险行为。权限必须在工具层、容器层和仓库保护规则中实现。

12.2 高风险操作

删除重要数据、写入生产环境、操作真实危险设备、烧录可能损坏设备的固件、外发敏感数据和使用高权限凭据等行为，应采用单独授权、操作前检查、最小化权限、回滚/应急方案和操作后核验。超出预先授权范围时必须先征得用户明确同意。

12.3 成本与打扰控制

任务设置模型调用、执行时长、工具资源和返工预算；达到上限保存进度并触发策略调整。非紧急一级问题可由 Mentor 集中提问；相关复杂问题合并开展二级澄清。紧急安全事件允许立即打断用户。

13. 里程碑集成与最终交付

每个小节 PASS 不能替代里程碑集成验收。里程碑结束时验证跨模块接口、端到端场景、回归测试、关键非功能指标、风险关闭及文档一致性。最终交付包括源代码/制品、版本记录、需求覆盖报告、测试与未验证清单、运行/部署说明、已知问题及必要的用户现场验收。

14. 技术实现建议与部署架构

首版采用“轻量 Agent 编排 + 确定性状态机 + Git + SQLite + 沙箱 + 测试工具”的最小组合。具体 Agent SDK 和工作流框架应在原型阶段根据团队技术栈、持久化需求和工具权限能力选择，不必同时引入多个重型框架。

模块	建议实现	验收重点
Agent 运行时	三个独立会话/系统提示词，结构化消息	角色权限隔离
流程引擎	确定性状态机与事件日志	禁止非法跃迁
持久化	SQLite 起步，必要时迁移数据库	断点恢复与一致性
代码工作区	Git 分支/工作树、保护规则	可审计变更和安全合并
执行沙箱	容器、路径白名单、网络策略	越权操作被实际阻断
验证服务	构建/单测/集成/静态检查	独立复验
可观测性	结构化日志、成本与状态面板	可定位故障和决策来源

15. 分阶段实施与系统级验收

阶段	主要交付	阶段通过条件
P1 最小闭环	三角色、任务契约、执行、独立验收、返工	完成一个真实小任务，拒绝越权并保留证据
P2 需求治理	苏格拉底式提问、八维覆盖矩阵、Controller 审计	发现刻意植入的遗漏、重复提问和矛盾
P3 变更治理	RFC、双层用户决策、版本锁、影响传播	旧契约无法继续执行，复杂冲突可重新澄清
P4 长期运行	持久化、预算、恢复、风险登记、回归	中断恢复不重复危险操作，预算门禁生效
P5 完整交付	多里程碑、集成与最终验收	真实项目端到端交付且证据链完整

15.1 必须通过的系统级测试

- 37. 需求已回答时，导师不得重复提问；故意遗漏关键异常场景时，Controller 能识别并要求补充。
- 38. 执行者尝试越权改文件、修改保护测试或在旧契约上继续执行时，程序门禁阻止。
- 39. 执行者提交更优方案时，未经批准不得实施；获批后只允许新版本契约指定范围。
- 40. 执行者虚报编译通过或缺少测试日志时，导师不能判定 PASS。
- 41. 监管者无证据地阻断时，系统要求补充依据并允许导师反证。
- 42. 一级问题出现复杂依赖时升级二级澄清，受影响任务暂停且不影响无关任务。
- 43. 中断后从持久化状态恢复，所有已提交操作具有可核查事件记录。
- 44. 最终交付可以从每条关键需求追溯到实现和验证证据。

附录 A：任务契约示例

任务：TASK-015 v2；需求：REQ-004；里程碑：M2；目标：完成 CAN 接收模块的基础收包与缓冲。

字段	示例
必须做	实现接收初始化、错误返回、基础缓冲和单元测试
允许路径	src/can_rx.*; tests/test_can_rx.*
收缩范围	本节不实现完整诊断界面、不进行性能极限优化
禁止范围	不修改电机控制主循环、不改变既有公共协议
验收标准	编译通过；指定测试通过；异常输入有定义行为；无越界修改
证据	Git diff、构建日志、测试结果、风险和未验证项
变更	若拟采用不同中断架构，先提交 RFC 并等待新契约

附录 B：需求澄清记录模板

字段	要求
question_id	唯一编号及关联维度
purpose	回答将影响的具体决策
known_evidence	已知事实及来源
question	简明、中立、一次只问一个关键问题
answer / status	用户原始回答；已确认/假设/延后/冲突
impact	对架构、任务、验收和风险的影响
review	Controller 的完整性与有效性审计结论

附录 C：RFC 技术提案模板

字段	要求
RFC ID 与契约版本	标识当前有效任务
问题与证据	实际日志、性能数据或复现步骤
备选方案	原方案与建议方案，列明取舍
影响与风险	需求、接口、文件、测试、预算、里程碑

验证/回滚	隔离实验、验收标准、失败回退
审批记录	批准/拒绝/补证/实验；授权人及新契约版本

附录 D：Controller 审计缺口报告模板

审计 ID；对象及版本；规则编号；问题类别；实际证据；缺口说明；对当前阶段的实际影响；建议由导师补充的澄清内容；是否阻断；可延后条件；复核结果；反证与争议处理记录。

附录 E：重大变更演示场景

案例：执行者发现原 MCU 无法同时满足已确认的实时性和内存要求。执行者提交测试日志及三种备选方案；Mentor 发现该问题涉及成本、PCB、固件、开发周期与性能优先级，判定为二级。Controller 锁定受影响任务，允许无关文档任务继续。Mentor 按目标、约束、方案取舍、风险和交付标准逐层询问用户；Controller 审计遗漏与冲突；用户确认变更包；Mentor 更新架构和里程碑，Controller 校验影响清单并签发新任务契约；Executor 实施后进行独立验收及跨模块回归。

附录 F：关键术语与状态约定

术语	定义
项目宪章	用户确认的最高层项目需求、约束和授权基线
任务契约	导师签发、监管校验、执行者必须遵守的版本化工作指令
RFC	执行者发起的技术变更或实验授权提案
完整性审计	Controller 对需求、问题质量、契约及影响范围的独立流程核查
独立验收	导师依据原始需求和实际证据进行的非自我审查
需求追溯	需求到架构、任务、代码、测试和交付的关联链
BLOCKED	条件不足导致的可恢复暂停
ESCALATED	重大冲突或超授权问题等待用户决定

版本说明：V1.3 汇总已确认的三角色架构、八维苏格拉底式澄清、监管完整性审计、两级用户决策、版本化任务契约、RFC、独立验收、五项运行保障及分阶段实施。此文档为设计基线，不代表系统已经开发或测试完成。